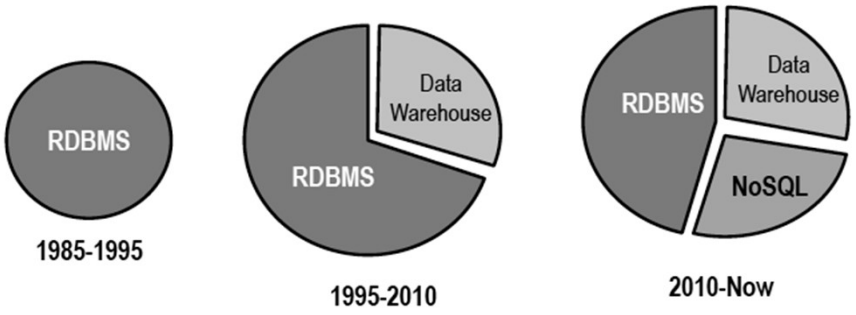
NoSQL

Vũ Tuyết Trinh
SoICT - HUST

1

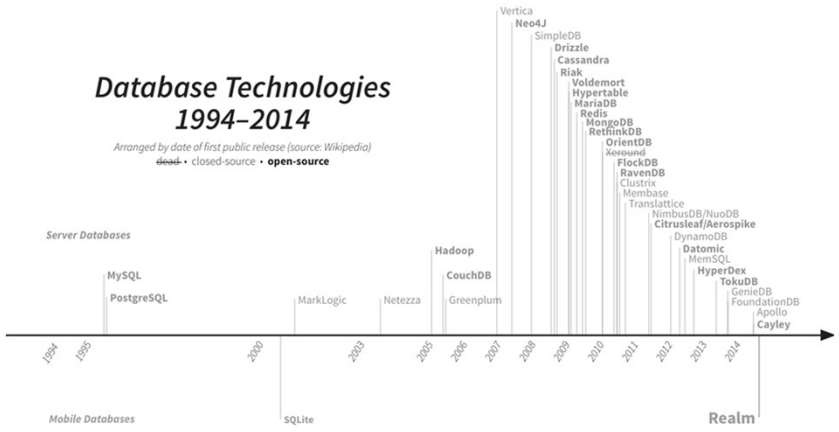
Eras of Databases



2

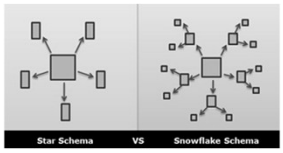
2

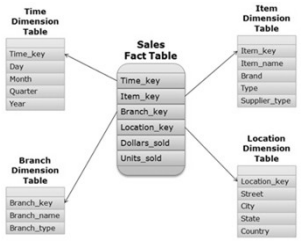
Eras of Databases



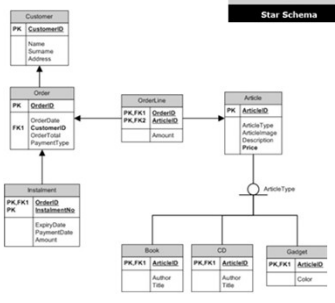
3

Before NoSQL

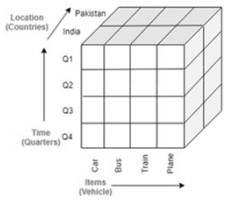




Star schema



OLTP

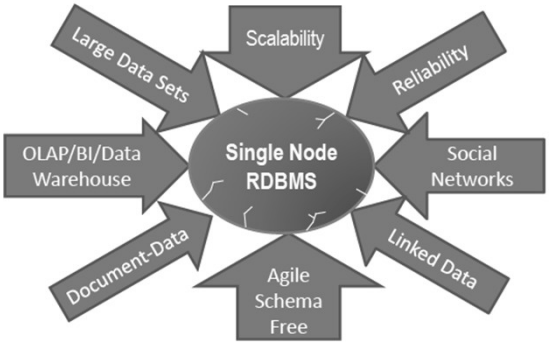


OLAP cube

4

4

RDBMS: one size fits all needs



5

5

ICDE 2005 conference



"One Size Fits All": An Idea Whose Time Has Come and Gone

Authors: Michael Stonebraker StreamBase Systems, Inc.
Ugur Cetintemel Brown University and StreamBase Systems, Inc.

2005 Article

Published in:
· Proceeding
ICDE '05 Proceedings of the 21st International Conference on Data Engineering
Pages 2-11

April 05 - 08, 2005
IEEE Computer Society Washington, DC, USA ©2005
table of contents ISBN:0-7695-2285-8 doi>10.1109/ICDE.2005.1

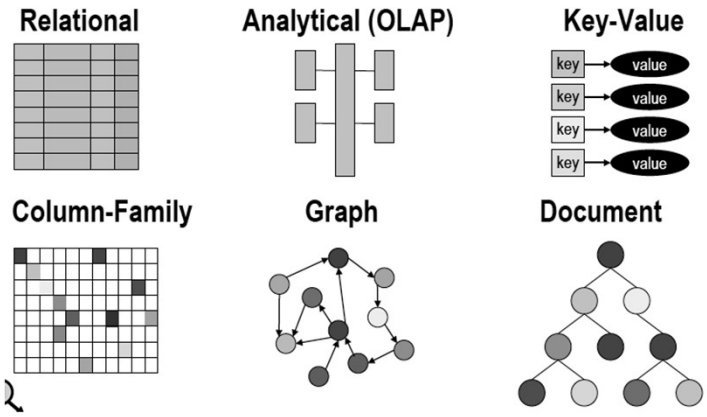
Bibliometrics
· Citation Count: 73
· Downloads (cumulative): 0
· Downloads (12 Months): 0
· Downloads (6 Weeks): 0

The last 25 years of commercial DBMS development can be summed up in a single phrase: "one size fits all". This phrase refers to the fact that **the traditional DBMS architecture (originally designed and optimized for business data processing) has been used to support many data-centric application**s with widely varying characteristics and requirements. In this paper, we argue that this concept is no longer applicable to the database market, and that the commercial world will fracture into a collection of independent database engines ...

6

6

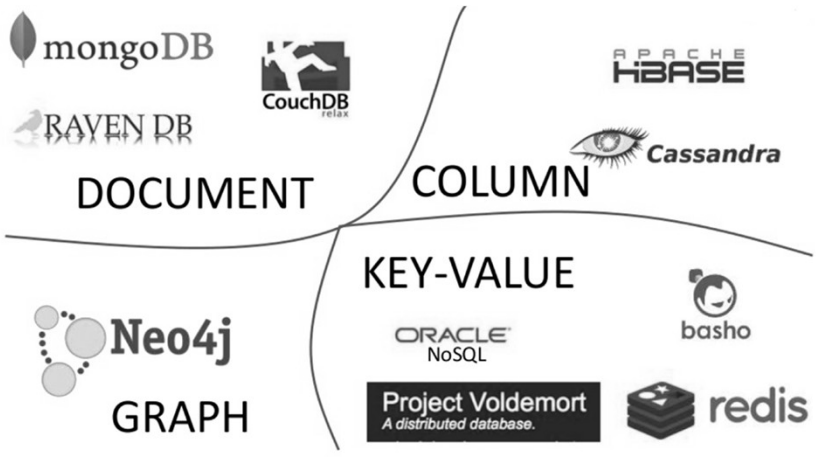
After NoSQL



7

7

NoSQL landscape



8

8

How to write a CV



9

9

Why NoSQL

- Web applications have different needs
  - Horizontal scalability – lowers cost
  - Geographically distributed
  - Elasticity
  - Schema less, flexible schema for semi-structured data
  - Easier for developers
  - Heterogeneous data storage
  - High Availability/Disaster Recovery
- Web applications do not always need
  - Transaction
  - Strong consistency
  - Complex queries

10

10

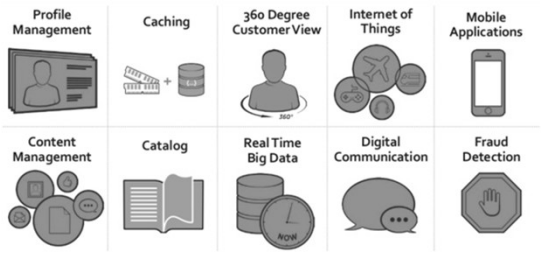
SQL vs NoSQL

SQL	NoSQL
Gigabytes to Terabytes	Petabytes(1kTB) to Exabytes(1kPB) to Zetabytes(1kEB)
Centralized	Distributed
Structured	Semi structured and Unstructured
Structured Query Language	No declarative query language
Stable Data Model	Schema less
Complex Relationships	Less complex relationships
ACID Property	Eventual Consistency
Transaction is priority	High Availability, High Scalability
Joins Tables	Embedded structures

11

NoSQL use cases

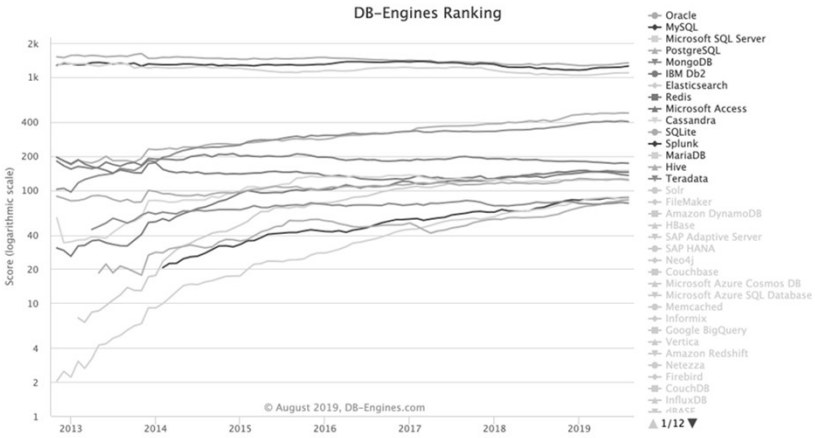
- Massive data volume at scale (Big volume)
  - Google, Amazon, Yahoo, Facebook – 10-100K servers
- Extreme query workload (Big velocity)
- High availability
- Flexible, schema evolution



12

12

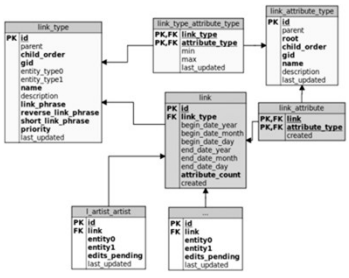
DB engines ranking according to their popularity (2019)



13

Relational data model revisited

- Data is usually stored in row by row manner (row store)
- Standardized query language (SQL)
- Data model defined **before** you add data
- Joins merge data from multiple tables
  - Results are tables
- **Pros:** Mature ACID transactions with fine-grain security controls, widely used
- **Cons:** Requires up front data modeling, does not scale well



Oracle, MySQL, PostgreSQL, Microsoft SQL Server, IBM DB/2

14

14

Key/value data model

- Simple key/value interface
  - GET, PUT, DELETE
- Value can contain any kind of data
- Super fast and easy to scale (no joins)
- Examples
  - Berkley DB, Memcache, DynamoDB, Redis, Riak

key	value
firstName	Bugs
lastName	Bunny
location	Earth





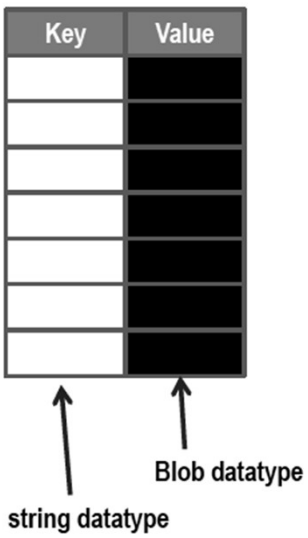
15

15

Key/value vs. table

- A table with two columns and a simple interface
  - Add a key-value
  - For this key, give me the value
  - Delete a key

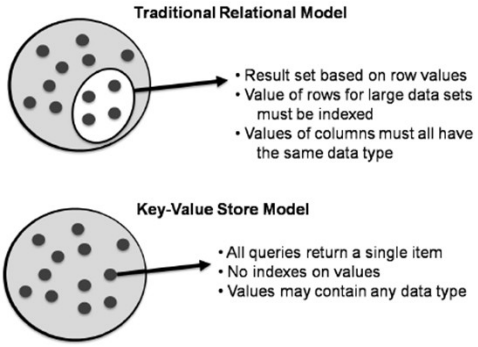




16

16

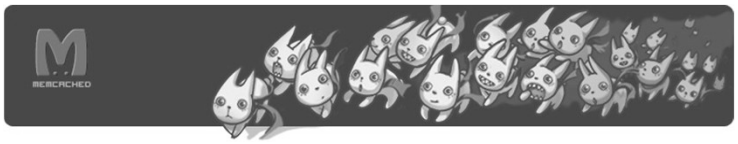
Key/value vs. Relational data model



17

17

Memcached



- Open source in-memory key-value caching system
- Make effective use of RAM on many distributed web servers
- Designed to speed up dynamic web applications by alleviating database load
  - Simple interface for highly distributed RAM caches
  - 30ms read times typical
- Designed for quick deployment, ease of development
- APIs in many languages

18

18

Redis

- Open source in-memory key-value store with optional durability
- Focus on high speed reads and writes of common data structures to RAM
- Allows simple lists, sets and hashes to be stored within the value and manipulated
- Many features that developers like expiration, transactions, pub/sub, partitioning



19

19

Amazon DynamoDB

- Scalable key-value store
- Fastest growing product in Amazon's history
- Focus on throughput on storage and predictable read and write times
- Strong integration with S3 and Elastic MapReduce



20

20

Riak

- Open source distributed key-value store with support and commercial versions by Basho
- A "Dynamo-inspired" database
- Focus on availability, fault-tolerance, operational simplicity and scalability
- Support for replication and auto-sharding and rebalancing on failures
- Support for MapReduce, fulltext search and secondary indexes of value tags
- Written in ERLANG

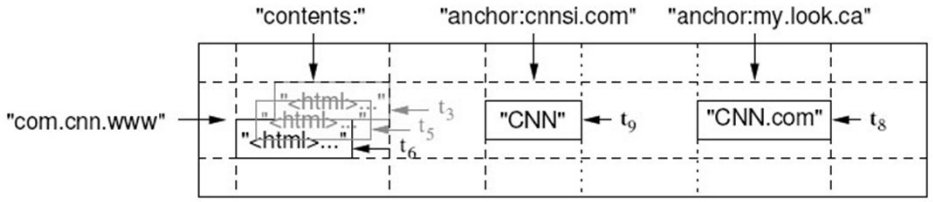
riak

21

21

Column family store

- Dynamic schema, column-oriented data model
- Sparse, distributed persistent multi-dimensional sorted map
- (row, column (family), timestamp) -> cell contents

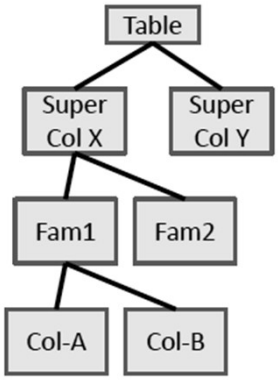


22

22

Column families

- Group columns into "Column families"
- Group column families into "Super-Columns"
- Be able to query all columns with a family or super family
- Similar data grouped together to improve speed

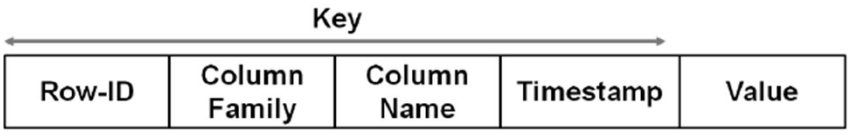


23

23

Column family data model vs. relational

- Sparse matrix, preserve table structure
  - One row could have millions of columns but can be very sparse
- Hybrid row/column stores
- Number of columns is extendible
  - New columns to be inserted without doing an "alter table"

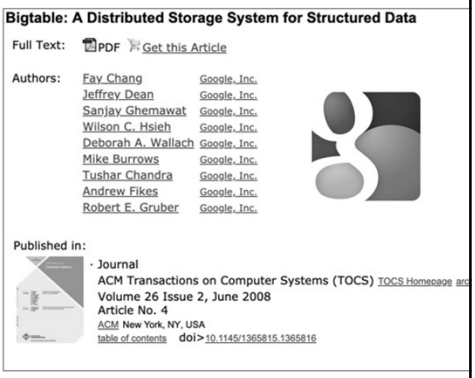


24

24

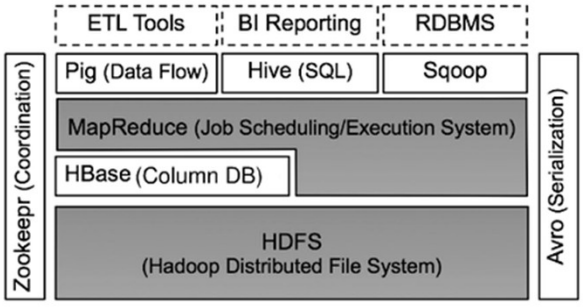
Bigtable

- ACM TOCS 2008
- Fault-tolerant, persistent
- Scalable
  - Thousands of servers
  - Terabytes of in-memory data
  - Petabyte of disk-based data
  - Millions of reads/writes per second, efficient scans
- Self-managing
  - Servers can be added/removed dynamically
  - Servers adjust to load imbalance



Apache Hbase

- Open-source Bigtable, written in JAVA
- Part of Apache Hadoop project



Apache Cassandra

- Apache open source column family database
- Supported by DataStax
- Peer-to-peer distribution model
- Strong reputation for linear scale out (millions of writes/second)
- Written in Java and works well with HDFS and MapReduce

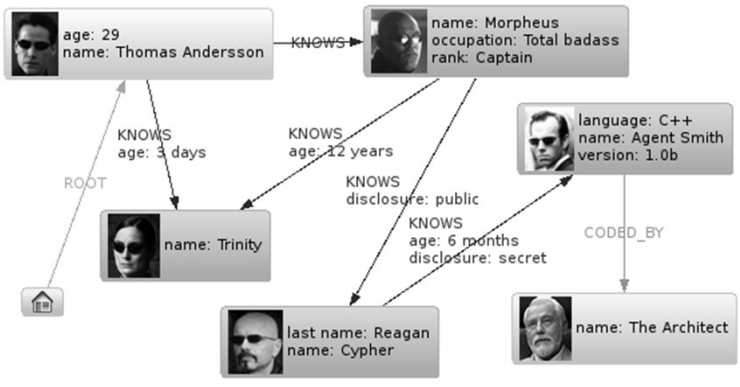


27

27

Graph data model

- Core abstractions: Nodes, Relationships, Properties on both



28

28

Graph database store

- A database stored data in an explicitly graph structure
- Each node knows its adjacent nodes
- Queries are really graph traversals

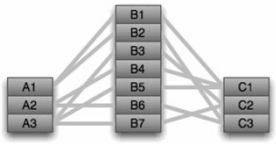


29

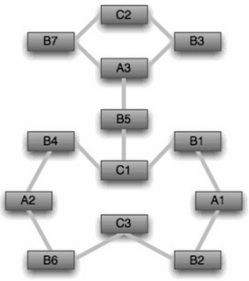
29

Compared to Relational Databases

Optimized for aggregation



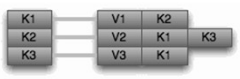
Optimized for connections



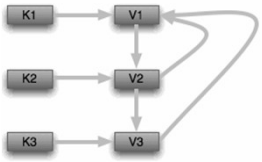
30

Compared to Key Value Stores

Optimized for simple look-ups



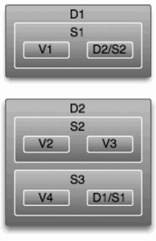
Optimized for traversing connected data



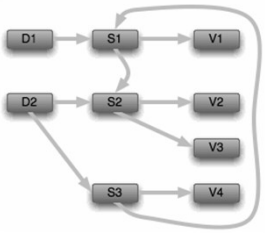
31

Compared to Document Stores

Optimized for “trees” of data

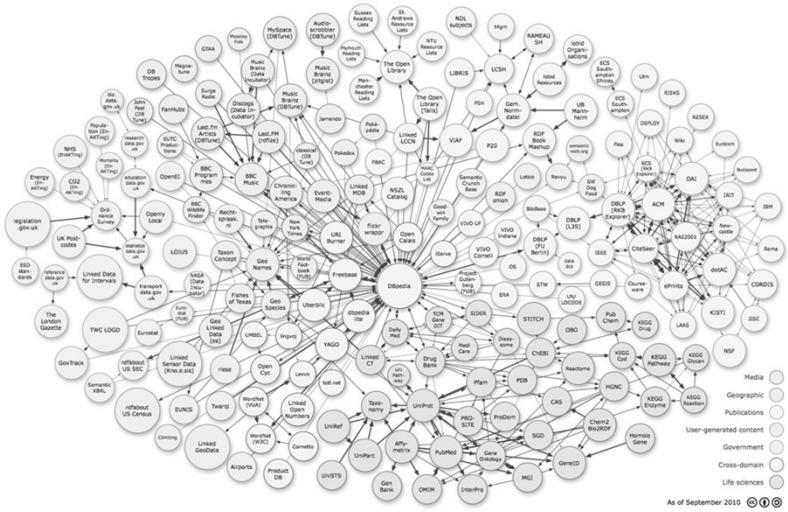


Optimized for seeing the forest and the trees, and the branches, and the trunks



32

Linking open data



LINKINGOPENDATA
W3C SWEO Community Project

33

33

Neo4j

- Graph database designed to be easy to use by Java developers
- Disk-based (not just RAM)
- Full ACID
- High Availability (with Enterprise Edition)
- 32 Billion Nodes, 32 Billion Relationships, 64 Billion Properties
- Embedded java library
- REST API

Neo4j

34

34

Document store

- Documents, not value, not tables
- JSON or XML formats
- Document is identified by ID
- Allow indexing on properties

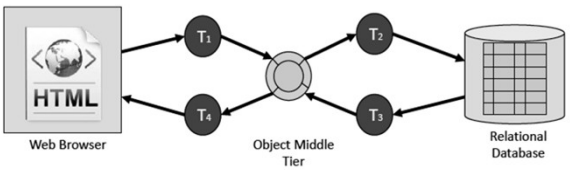
```
{
  person: {
    first_name: "Peter",
    last_name: "Peterson",
    addresses: [
      {street: "123 Peter St"},
      {street: "504 Not Peter St"}
    ],
  }
}
```

35

35

Relational data mapping

- T1–HTML into Objects
- T2–Objects into SQL Tables
- T3–Tables into Objects
- T4–Objects into HTML

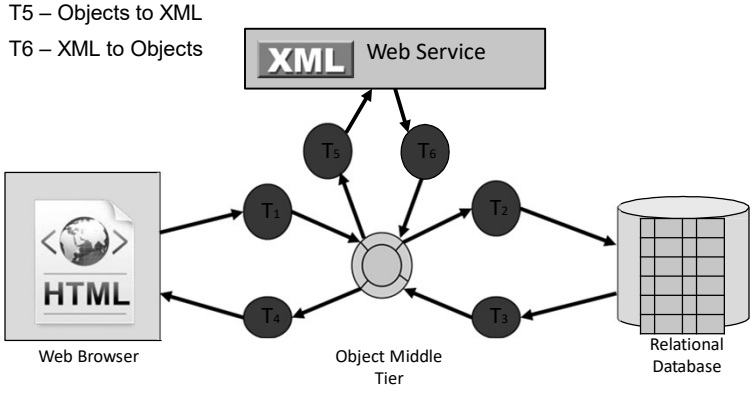


36

36

Web Service in the middle

- T1 – HTML into Java Objects
- T2 – Java Objects into SQL Tables
- T3 – Tables into Objects
- T4 – Objects into HTML
- T5 – Objects to XML
- T6 – XML to Objects



37

37

Discussion

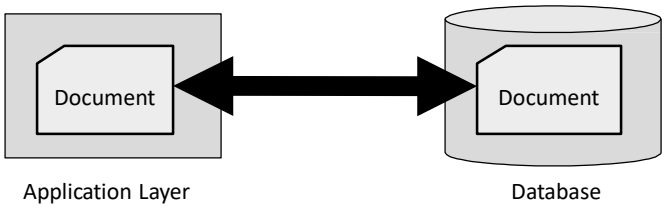
- Object-relational mapping has become one of the most complex components of building applications today
  - Java Hibernate Framework
  - JPA
- To avoid complexity is to keep your architecture very simple

38

38

Document mapping

- Documents in the database
- Documents in the application
- No object middle tier
- No "shredding"
- No reassembly
- Simple!



39

39

MongoDB

- Open Source JSON data store created by 10gen
- Master-slave scale out model
- Strong developer community
- Sharding built-in, automatic
- Implemented in C++ with many APIs (C++, JavaScript, Java, Perl, Python etc.)

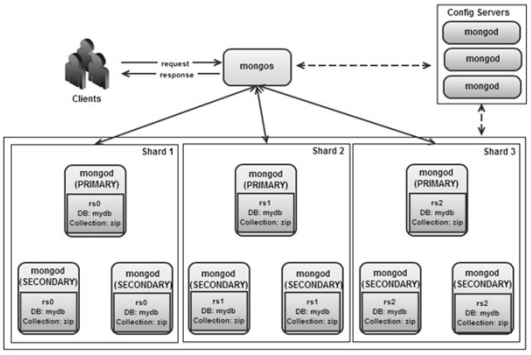
mongoDB

40

40

MongoDB architecture

- Replica set
  - Copies of the data on each node
  - Data safety
  - High availability
  - Disaster recovery
  - Maintenance
  - Read scaling
- Sharding
  - “Partitions” of the data
  - Horizontal scale



41

Apache CouchDB

- Apache project
- Open source JSON data store
- Written in ERLANG
- RESTful JSON API
- B-Tree based indexing, shadowing b-tree versioning
- ACID fully supported
- View model
- Data compaction
- Security



42

42

CAP Theorem

Consistency, Availability, and Partition Tolerance

43

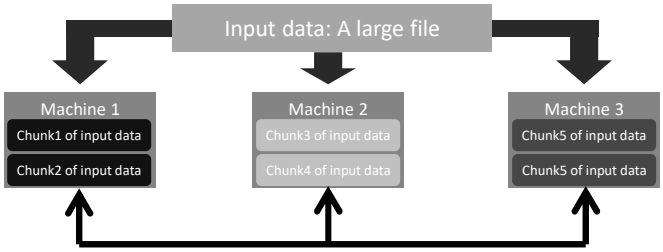
Scaling Traditional Databases

- Traditional RDBMSs can be either scaled:
  - Vertically (or Up)
    - Can be achieved by hardware upgrades (e.g., faster CPU, more memory, or larger disk)
    - Limited by the amount of CPU, RAM and disk that can be configured on a single machine
  - Horizontally (or Out)
    - Can be achieved by adding more machines
    - Requires database sharding and probably replication
    - Limited by the Read-to-Write ratio and communication overhead

44

Data sharding

- Data is typically sharded (or striped) to allow for concurrent/parallel accesses
- Will it scale for complex query processing?

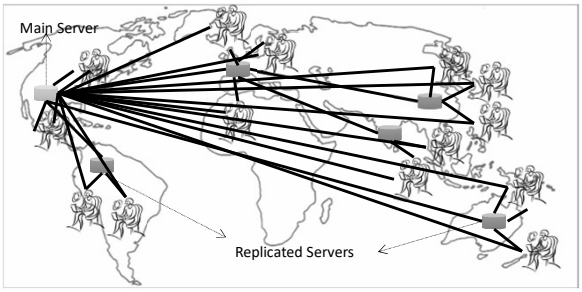


E.g., Chunks 1, 3 and 5 can be accessed in parallel

45

Data replication

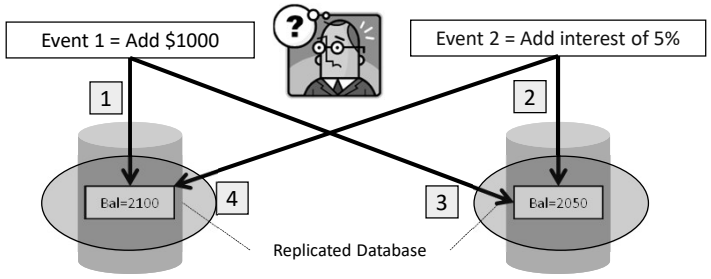
- Replicating data across servers helps in:
  - Avoiding performance bottlenecks
  - Avoiding single point of failures
  - And, hence, enhancing scalability and availability



46

But, Consistency Becomes a Challenge

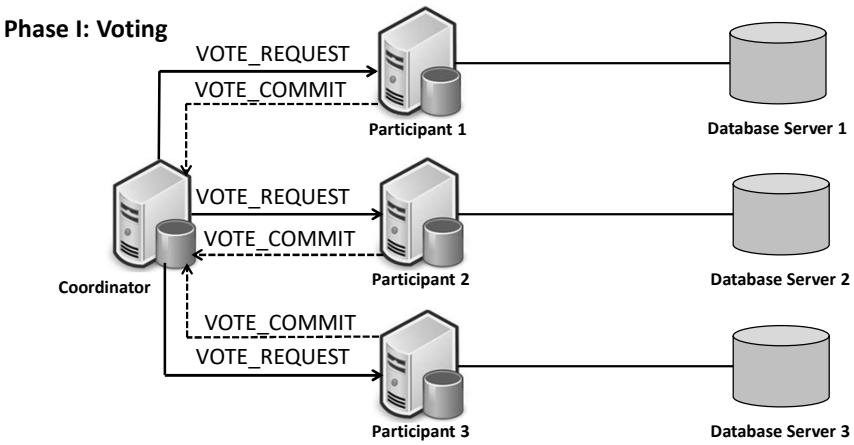
- An example:
  - In an e-commerce application, the bank database has been replicated across two servers
  - Maintaining consistency of replicated data is a challenge



47

The Two-Phase Commit Protocol

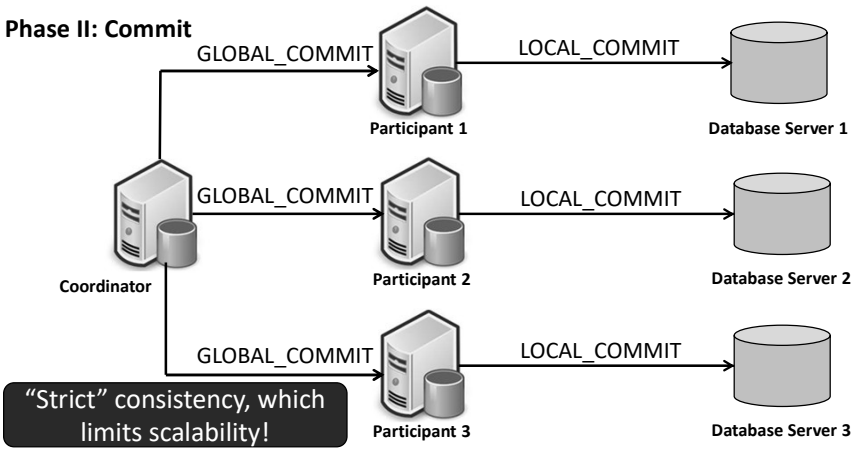
- The two-phase commit protocol (2PC) can be used to ensure atomicity and consistency



48

The Two-Phase Commit Protocol

- The two-phase commit protocol (2PC) can be used to ensure atomicity and consistency



49

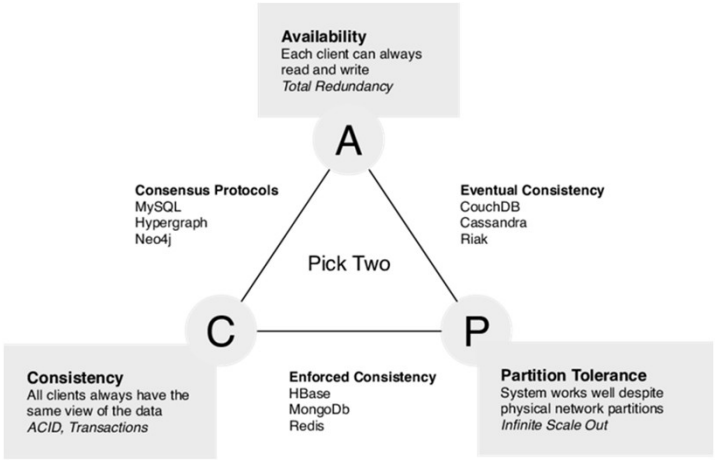
The CAP Theorem

- The limitations of distributed databases can be described in the so called the CAP theorem
  - Consistency: every node always sees the same data at any given instance (i.e., strict consistency)
  - Availability: the system continues to operate, even if nodes in a cluster crash, or some hardware or software parts are down due to upgrades
  - Partition Tolerance: the system continues to operate in the presence of network partitions

CAP theorem: any distributed database with shared data, can have *at most two* of the three desirable properties, C, A or P. These are trade-offs involved in distributed system by Eric Brewer in PODC 2000.

50

CAP Theorem

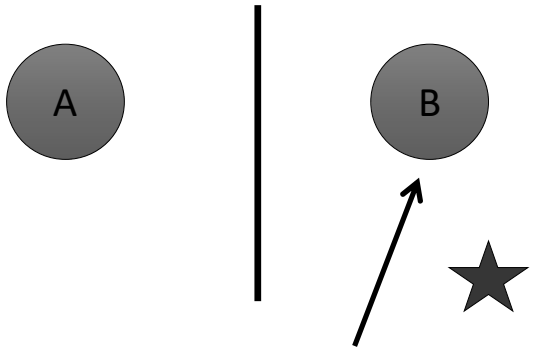


51

51

CAP Theorem: Proof

- A simple proof using two nodes:

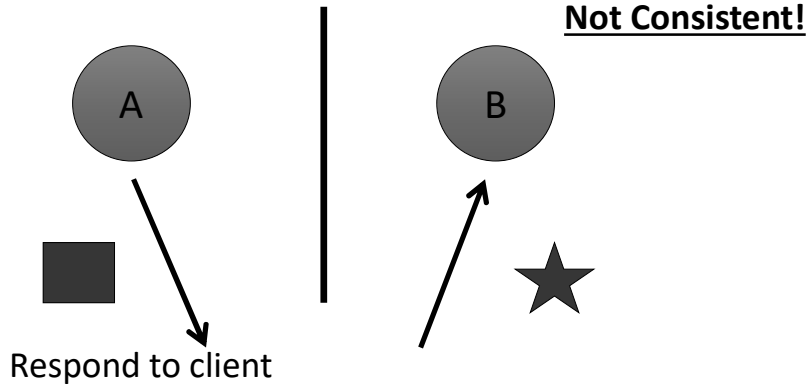


52

52

CAP Theorem: Proof

- A simple proof using two nodes:

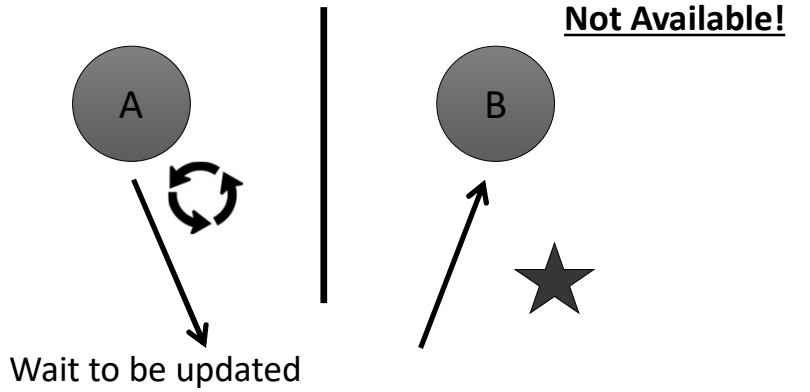


53

53

CAP Theorem: Proof

- A simple proof using two nodes:

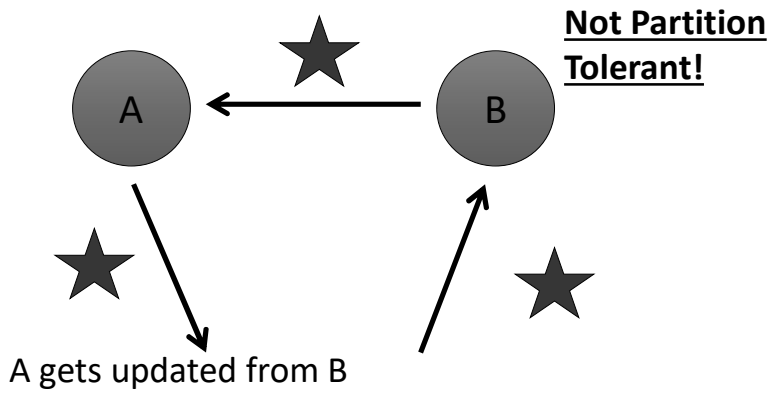


54

54

CAP Theorem: Proof

- A simple proof using two nodes:



55

55

Scalability of relational databases

- The Relational Database is built on the principle of **ACID** (Atomicity, Consistency, Isolation, Durability)
- It implies that a truly distributed relational database should have **availability, consistency and partition tolerance**.
- Which unfortunately is **impossible** …

56

56

Large-Scale Databases

- When companies such as Google and Amazon were designing large-scale databases, 24/7 Availability was a key
  - A few minutes of downtime means lost revenue
- When horizontally scaling databases to 1000s of machines, the likelihood of a node or a network failure increases tremendously
- Therefore, in order to have strong guarantees on Availability and Partition Tolerance, they had to sacrifice “strict” Consistency (implied by the CAP theorem)

57

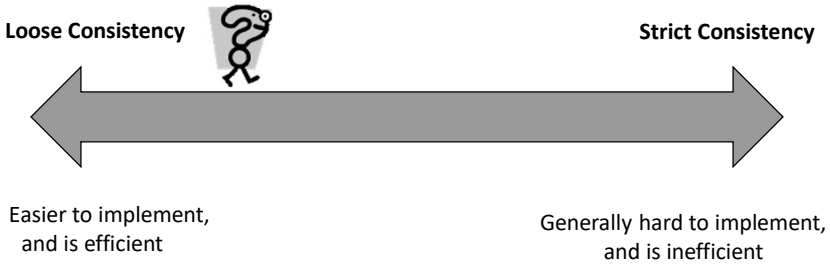
Trading-Off Consistency

- Maintaining consistency should balance between the strictness of consistency versus availability/scalability
  - Good-enough consistency depends on your application

58

Trading-Off Consistency

- Maintaining consistency should balance between the strictness of consistency versus availability/scalability
  - Good-enough consistency depends on your application



Easier to implement, and is efficient

Generally hard to implement, and is inefficient

59

The BASE Properties

- The CAP theorem proves that it is impossible to guarantee strict Consistency and Availability while being able to tolerate network partitions
- This resulted in databases with relaxed ACID guarantees
- In particular, such databases apply the BASE properties:
  - Basically Available: the system guarantees Availability
  - Soft-State: the state of the system may change over time
  - Eventual Consistency: the system will eventually become consistent

60

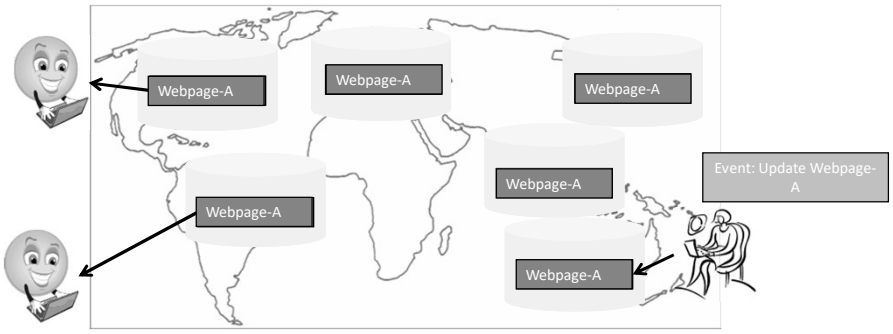
Eventual Consistency

- A database is termed as Eventually Consistent if:
  - All replicas will gradually become consistent in the absence of new updates

61

Eventual Consistency

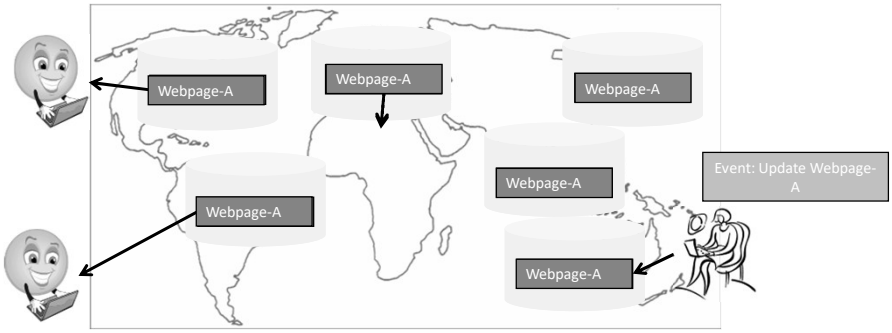
- A database is termed as Eventually Consistent if:
  - All replicas will gradually become consistent in the absence of new updates



62

Read-after-write consistency (eg. Amazon S3)

- But, what if the client accesses the data from different replicas?



Protocols like Read Your Own Writes (RYOW) can be applied!

63

References

- Han, Jing, et al. "Survey on NoSQL database." *2011 6th international conference on pervasive computing and applications*. IEEE, 2011.
- Sivasubramanian, Swaminathan. "Amazon dynamoDB: a seamlessly scalable non-relational database service." *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. 2012.
- Chang, Fay, et al. "Bigtable: A distributed storage system for structured data." *ACM Transactions on Computer Systems (TOCS)* 26.2 (2008): 1-26.
- Iordanov, Borislav. "HyperGraphDB: a generalized graph database." *International conference on web-age information management*. Springer, Berlin, Heidelberg, 2010.

64

References (2)

- Gilbert, Seth, and Nancy Lynch. "Perspectives on the CAP Theorem." *Computer* 45.2 (2012): 30-36.
- Kleppmann, Martin. "A Critique of the CAP Theorem." *arXiv preprint arXiv:1509.05393* (2015).
- Brewer, Eric. "CAP twelve years later: How the" rules" have changed." *Computer* 45.2 (2012): 23-29.
- Chandra, Deka Ganesh. "BASE analysis of NoSQL database." *Future Generation Computer Systems* 52 (2015): 13-21.

65

Q & A

66

66